LESSON 2

GPU・ストレージ・マザーボード

目標：絵が描かれる仕組みと、データが運ばれる道のりを説明できる

GPU

CPU と GPU は、得意なことが正反対

CPU：少数精鋭

- 数個〜数十個の、賢くて速いコア
- if 文だらけの複雑な判断が得意
- 順番どおりに進める処理が得意
- たとえ：**数学の先生が数人**

GPU：人海戦術

- 上位品では**2万個を超える**単純な計算器
- 同じ計算を、大量のデータに一斉にかける
- ばらばらな判断は苦手
- たとえ：**計算ドリルを解く小学生が数万人**

POINT 画面の絵は「全部の画素に同じ計算をする」仕事なので、GPU に向いている。同じ理由で、**AI の学習**も GPU が担っている。

GPU

1秒間に、何回の色を計算しているか

フル HD（1920×1080）

約207万

画素

4K（3840×2160）

約829万

画素（フル HD の4倍）

4K を 60fps で

約5億

画素ぶんの色を1秒で計算

- 1画素ごとに、テクスチャを読み、光の当たり方を計算し、影を判定する
- しかも計算の中身は**どの画素でもほぼ同じ** → 大量の計算器で一斉に処理できる
- 解像度を上げると、GPU の仕事は画素の数に比例して増える

GPU

3D の絵ができるまで（レンダリングパイプライン）

1
頂点処理
モデルの頂点を、3D の位置から画面上の位置に変換する（頂点シェーダー）

2
ラスタライズ
三角形が画面のどの画素を覆うかを割り出す

3
画素の色
テクスチャと光から、1画素ずつ色を決める（フラグメントシェーダー）

4
重ね合わせ
手前の物を優先し、半透明を混ぜ、完成した絵をためる

ドローコール：CPU から GPU への「これを描いて」

描く物ごとに CPU が GPU へ命令を出す。**数が多すぎると、GPU に余裕があっても CPU 側が詰まる**。Unity のバッチングや GPU インスタンシングは、この回数を減らす工夫。

POINT Unity の Shader Graph で作っているのは、1 と 3 の中身。マテリアルの設定は、GPU で動く小さなプログラムへの入力です。

ゲームとの関係

fps とフレーム時間

目標の fps	1フレームの持ち時間
30 fps	33.3 ミリ秒
60 fps	16.7 ミリ秒
120 fps	8.3 ミリ秒
144 fps	6.9 ミリ秒
240 fps	4.2 ミリ秒

- fps は「1秒に何枚描けたか」。開発では**1枚に何ミリ秒かかったか**で考える
- CPU と GPU は同時に別のフレームを処理している（CPU が次、GPU が今）
- だから**遅い方**が fps の上限を決める
- 60fps から 120fps にするには、持ち時間を**半分**にする必要がある

注意 平均が 60fps でも、たまに 50 ミリ秒かかるフレームがあると**カクッ**と感じる（スタッター）。体感を決めるのは平均ではなく、**一番遅いフレーム**。

GPU

VRAM：GPU 専用の作業机

グラフィックボードに載っている専用メモリ。描画に使うものはすべてここに置く。

- テクスチャ（画像）
- メッシュ（モデルの形）
- 描いている途中・描き終わった絵
- 影の計算用の画像（シャドウマップ）

テクスチャ1枚の大きさ

4096 × 4096 画素 × 4 バイト
＝ **64 MB**（圧縮なし）

縮小版（ミップマップ）を含めると約 1.33 倍。GPU 用の圧縮をかけると 1/4 程度に減る。

注意 VRAM からあふれると、メインメモリに置いて**マザーボード経由で取りに行く**。通り道が何倍も細いので、fps が急に落ちたりカクついたりする。

POINT Unity のテクスチャの Max Size と Compression の設定は、**VRAM の使用量を決める設定**。必要以上に大きな画像を入れない。

GPU の選び方

スペック表の読み方

項目	意味
シェーダー数	計算器の数。CUDA コア（NVIDIA）など
ブーストクロック	計算器が動く速さ
VRAM 容量	置ける素材の量。足りないと急に遅くなる
メモリバス幅	VRAM との通り道の太さ（128bit、256bit など）
メモリ帯域	1秒に VRAM とやりとりできる量
消費電力（TGP）	必要な電源の大きさと、出る熱の量

帯域を計算する

バス幅 256bit、速度 28Gbps なら

256 × 28 ÷ 8
＝ 896 GB/秒

メインメモリ（約 96GB/秒）の**約9倍**

POINT 数字はメーカーや世代をまたいでは比べにくい。**同じゲームでの fps 比較（ベンチマーク）**で判断するのが確実。

GPU の選び方

3つのメーカーと型番の読み方

NVIDIA

RTX 5070 Ti

50＝世代、70＝グレード（50〜90）、Ti / SUPER＝同じグレードの上位版

AMD

RX 9070 XT

9＝世代、070＝グレード、XT＝上位版

Intel

Arc B580

B＝世代（A→B→C の順）、580＝グレード

- グレードの数字が大きいほど上位。**世代が新しい 60 番台が、古い 80 番台に負けることも勝つこともある**
- 同じ GPU でも、ボードを作る会社（ASUS、MSI など）ごとに冷却やクロックが少し違う
- ゲーム開発では、自分の PC だけでなく**遊ぶ人の多い GPU**で動くかを考える

GPU の新しい機能

ラスタライズとレイトレーシング

ラスタライズ（従来）

- 三角形を画面に写して、上から塗っていく
- とても速い
- 反射や影は、別の計算で**それらしく見せている**

レイトレーシング

- カメラから光線を飛ばし、何に当たって、どこから光が来るかを追う
- 鏡の反射、影、光の回り込みが正確
- とても重い。専用の回路（RT コア）で速くしている

POINT 今のゲームは、ほとんどを**ラスタライズ**で描き、反射や影など一部だけに**レイトレーシング**を使う組み合わせが主流。

GPU の新しい機能

アップスケーリングとフレーム生成

アップスケーリング

- 低い解像度で描き、AI などで高い解像度に**引き伸ばす**
- 画素の計算が減るので fps が上がる
- 前のフレームの情報も使い、細部を補う

フレーム生成

- 実際に描いた絵と絵の**間の絵を AI が作って**はさむ
- 画面の fps の数字は大きく上がる
- ゲームの処理そのものは速くなっていない

メーカー	NVIDIA	AMD	Intel
技術の名前	DLSS	FSR	XeSS

注意 フレーム生成で 120fps と表示されても、**操作への反応は元の 60fps のまま**（むしろ少し遅れる）。動きはなめらかに見えるが、反応速度が大事なゲームでは効果が薄い。

ソフトとの境目

ゲームから GPU までの階段

あなたのゲーム（C# スクリプト・シーン）

ゲームエンジン（Unity）

グラフィックス API（DirectX・Vulkan・Metal）

GPU ドライバ（メーカーが配布）

GPU（ハードウェア）

動かす先	主な API
Windows	DirectX 11 / 12、Vulkan
Mac・iPhone	Metal
Android	Vulkan、OpenGL ES
家庭用ゲーム機	各社の専用 API

POINT

Unity が API の違いを吸収してくれるので、同じゲームをいろいろな機種に出せる。**GPU ドライバの更新**で fps が上がったり、表示のバグが直ったりするのは、一番下の階段が変わるから。

ストレージ

HDD と SSD

項目	HDD	SSD
記録のしかた	回る円盤に磁気で書く	半導体のチップに電子で書く
読み書きの速さ	毎秒 100〜250 MB ほど	毎秒 500 MB〜十数 GB
ばらばらな読み出し	とても遅い	速い
衝撃・音	衝撃に弱い。動作音がする	衝撃に強い。無音
1TB あたりの値段	安い	高め
向いている用途	大量の保存、バックアップ	OS、アプリ、ゲーム

POINT

どちらも**電源を切っても消えない**（不揮発）。今の PC は SSD に OS とゲームを入れ、大容量の保存用に HDD を足す組み合わせが多い。

ストレージ

HDD はなぜ遅いのか

- 中には**円盤（プラッタ）**が入っていて、1分間に 7200 回転する
- 読み書きする**ヘッド**が、円盤の上を数ナノメートルの高さで浮いて動く
- レコードプレーヤーの針を、ものすごい速さで動かしているイメージ

データを読むまでの待ち時間

① **シーク**：ヘッドを目的の位置まで動かす（数ミリ秒）

② **回転待ち**：目的の場所がヘッドの下に来るまで待つ

60秒 ÷ 7200回転 ＝ 1回転 8.3ミリ秒
平均で半回転 ＝ 約 4.2ミリ秒

POINT 大きなファイルを順に読むのは得意だが、**小さなファイルをあちこちから読む**と、そのたびに①②の待ちが入る。ゲームのロードは後者が多い。

注意 動いているときに強い衝撃を与えると、ヘッドが円盤にぶつかって**壊れる**ことがある。電源が入った PC や外付け HDD を動かさない。

ストレージ

SSD の中身：NAND フラッシュ

小さな部屋（セル）に**電子を閉じ込めて**記録する。電子は電源を切っても逃げないので、データが残る。

方式	1セルに記録する量	速さ・書き換え耐久	容量あたりの安さ
SLC	1 bit	最も高い	最も高価
MLC	2 bit	高い	高価
TLC	3 bit	標準（今の主流）	標準
QLC	4 bit	低め	安い

書き換えに寿命がある

保証は TBW（何 TB まで書けるか）で表す。ふつうの使い方なら何年も持つ。

均等に使って長持ち

中の制御チップが、同じ場所ばかり書かないよう散らしている。

上書きができない

一度消してから書く。消すのはまとまった単位なので、空きを整理する仕組み（TRIM）が要る。

マザーボードの上にあるもの

CPU ソケット

CPU を取り付ける台。形が合う CPU しか載らない

メモリスロット

メモリを挿す。DDR4 用か DDR5 用かが決まっている

PCIe スロット

グラフィックボードなどの拡張カードを挿す

M.2 スロット

板状の SSD を直接挿す

チップセット

USB や SATA など、周りの機器の交通整理役

VRM

CPU に合った電圧の電気を作る回路

背面 I/O

USB、LAN、音声、映像の端子が並ぶ

BIOS チップ

電源を入れて最初に動くプログラムが入っている

POINT マザーボードは「何を、いくつ、どの速さでつなげるか」を決める。**CPU とメモリの種類は、マザーボードを選んだ時点で決まる。**

マザーボード

BIOS / UEFI：OS より先に動くプログラム

マザーボードのチップに入っていて、電源を入れると最初に動く。パーツを確かめ、OS を呼び出すまでが仕事。起動直後に **Del** や **F2** キーで設定画面に入れる。

よく触る設定	内容
XMP / EXPO	メモリを表記どおりの速度で動かす（1時間目）
起動の順番	どのストレージや USB メモリから OS を起動するか
ファンの制御	温度に応じた回転数の決め方
仮想化（VT-x / SVM）	Android エミュレーターなど、PC の中で別の PC を動かす機能
セキュアブート・TPM	不正なプログラムの起動を防ぐ。Windows 11 の必須条件

注意 BIOS の更新中に電源が切れると、**PC が起動しなくなる**ことがある。新しい CPU に対応させるために必要な場面もあるが、手順を守って慎重に行う。

起動の流れ

電源ボタンからログイン画面まで

1. **電源ユニット**が電気を送り始め、電圧が安定したことをマザーボードに知らせる
2. **UEFI** が動き出し、CPU・メモリ・GPU などが正しく動くか検査する（POST）
3. 設定された順番で**起動するストレージ**を探す
4. **ブートローダー**（OS を呼び出す小さなプログラム）を読み込む
5. **Windows の中心部分（カーネル）**とドライバをメモリに読み込む
6. ログイン画面が表示される

意外 Windows の「シャットダウン」は、初期設定では 5 の状態を保存して切る（高速スタートアップ）。**不調を直したいときは「再起動」**を選ぶと、本当に最初から起動し直す。

取り扱い

PC の中を触るときの注意

電源ケーブルを抜く

シャットダウン後も、マザーボードには電気が流れている。作業前に必ず抜く。

静電気を逃がす

パーツに触る前に、ケースの金属部分に触れる。冬の乾燥した日は特に注意。

端子に触れない

メモリや GPU は、金色の端子ではなく縁を持つ。

ピンを曲げない

CPU ソケットの細かいピンは、1本曲がるだけで動かなくなる。

奥まで挿す

メモリや GPU は「カチッ」と留め具がかかるまで。半挿しは不調の定番。

電源ユニットは開けない

中の部品（コンデンサ）は、コンセントを抜いた後も電気をためていて危険。

POINT 学校の PC は勝手に開けない。自分の PC でも、メーカー製は分解すると保証が切れることがある。

LESSON 3

AI とクラウド

目標：GPU が AI を支える理由と、手元にないパーツでゲームが動く仕組みを説明できる

AI とハードウェア

なぜ AI には GPU が使われるのか

AI の中身は「掛けて足す」

答え ＝ 重み1 × 入力1
＋ 重み2 × 入力2
＋ …（これを何十億回）

1つ1つは単純な計算。ただし**量がけた違い**に多い。

それは GPU が得意な形

- 中身が**同じ計算**を、**大量に一斉に**行う
- 画素の色を一気に計算するのと、まったく同じ構造（2時間目の話）
- CPU の数コアより、GPU の数千の計算器が圧倒的に速い

2012年 画像認識の大会で、**ゲーム用の GPU 2枚**で学習した AI が圧勝。ここから今の AI ブームが始まった。

POINT **ゲームの絵を描くために進化した GPU** が、たまたま AI の計算にもぴったりだった。ゲーム業界が育てた部品が、AI の時代を支えている。

AI とハードウェア

「学習」と「推論」で、必要なパーツが違う

	学習（AI を作る）	推論（AI を使う）
やること	大量のデータから「重み」を少しずつ調整する	決まった重みで、答えを1回出す
計算の量	とてつもなく多い	学習よりずっと少ない
どこで	データセンター。GPU を数千〜数万台つなぐ	データセンター、または手元の PC・スマホ
かかる時間	数週間〜数か月	一瞬〜数秒

AI 用の専用回路

最近の GPU には「掛けて足す」専用の回路（NVIDIA ならTensor コア）が入っている。

精度を落として速く

AI は多少の誤差に強い。double ではなく、もっと小さい数で計算して速度を稼ぐ。

注意 手元で AI を動かすとき、効くのは GPU の速さより **VRAM の容量**。AI の重みがすべて VRAM に入らないと、極端に遅くなる。

AI とハードウェア

NPU：CPU に載り始めた AI 専用の装置

装置	得意なこと	電力	AI での使われ方
CPU	複雑な判断を1つずつ速く	中くらい	小さな AI なら動かせる
GPU	同じ計算を大量に一斉に	多い	重い AI を一気に
NPU	AI の「掛けて足す」だけ	とても少ない	軽い AI を、ずっと動かし続ける

- スマホには何年も前から載っている（顔認証、写真の補正）
- PC では Intel Core Ultra、AMD Ryzen AI などに搭載
- 使い道：ビデオ通話の背景ぼかし、雑音消し、文字起こし

性能の単位

TOPS

1秒間に何兆回計算できるか
（40 TOPS ＝ 1秒に40兆回）

POINT　ゲームの絵や重い AI は GPU、**省電力でずっと動かす軽い AI は NPU**。CPU の中に、また新しい専門家が増えた。

ゲームの「AI」は2種類ある

ゲーム AI（昔からある）

- 敵の動き、追跡、経路探索
- 中身は **if 文やステートマシン**。人がルールを書く
- 動かすのは **CPU**（メインスレッド）

機械学習の AI（新しい）

- データから学んだ「重み」で答えを出す
- 中身は**大量の掛け算と足し算**。ルールは人が書かない
- 動かすのは **GPU や NPU**

機械学習の AI の使われ方	例
遊ぶ人の PC の中で	低い解像度の絵を AI で高画質にする（DLSS など、2時間目のアップスケール）
開発する人の PC・クラウドで	コードの補完、画像や音声の素材づくり、テストプレイの自動化

POINT 同じ「AI」という言葉でも、**中身も、動かすパーツもまったく違う**。授業で作ってきた敵の動きは、CPU が動かすゲーム AI のほう。

クラウドとサーバー

クラウドゲーミング：PC は遠くにある

1
手元
ボタンの入力だけを送る

2
データセンター
本物の CPU と GPU がゲームを動かし、絵を描く

3
圧縮
描いた絵を動画として小さくする

4
インターネット
動画を手元に送る

5
手元
動画を展開して表示する

- 1時間目の「ゲームを起動すると中で何が起きるか」の 3〜4 が、**遠くの建物の中**で起きている
- 手元の機械は、動画を再生できれば十分。スマホやテレビでも遊べる
- 例：Xbox Cloud Gaming、GeForce NOW、PlayStation Plus のクラウドストリーミング

POINT パーツは**手元になくても動く**。ゲームを動かす仕組みはそのままで、置き場所が変わっただけ。

クラウドとサーバー

クラウドゲーミングの弱点：遅れと回線

光の速さでも

約5ミリ秒

東京⇄大阪の往復（光ファイバー）

必要な回線の速さ（フル HD）

約25 Mbps

が目安

1時間遊ぶと

約11 GB

の通信量

- 入力が届く → 処理する → 圧縮する → 送る → 展開する。**どの段階でも少しずつ遅れる**
- 手元のゲーム機で遊ぶより、反応が数十ミリ秒ほど遅くなることが多い
- 回線が混むと、画質が落ちたり、一瞬止まったりする

	ジャンルの例
向いている	RPG、シミュレーション、パズル、ターン制のゲーム
苦手	格闘ゲーム、音楽ゲーム、FPS など一瞬の反応が勝負のゲーム

オンラインゲームを支えるサーバーの中身

パーツ	ゲーミング PC	サーバー
CPU	1コアが速いものを重視	コアが多いものを重視（数十〜百以上）
メモリ	16〜32GB	数百 GB 以上。間違いを自動で直す種類（ECC）
GPU	必須	載っていないことも多い（画面を描かないから）
電源	1台	2台。片方が壊れても止まらない
形	机の横に置くケース	薄い箱を棚に何十台も重ねる
動く時間	使うときだけ	24時間365日

こうしたサーバーを何万台も並べた建物が**データセンター**。電気と冷却に、町ひとつ分ほどのエネルギーを使うものもある。

POINT

Unity には、**画面を描かないサーバー専用のビルド**（Dedicated Server）がある。GPU のないサーバーで、ゲームの計算だけを動かすため。